

RIPPLEGUARD

An Explainable Software Supply-Chain Digital Twin

MAP Build the dependency ecosystem	TRACE Find reachable propagation paths
SIMULATE Explore what happens after compromise	BREAK Find the smallest high-impact mitigation

Hackathon Problem Area

Open Source Supply Chains: The Ripple Effect

SDG 9 — Industry, Innovation and Infrastructure

This document explains the selected problem statement, the gaps identified in the supplied research, and a detailed proposed solution designed to satisfy the Round 1 requirements while going beyond them through interactive compromise simulation, explainable risk, and mitigation optimization.

Prepared as a solution concept and implementation blueprint

1. Executive Summary

Modern applications rarely depend on a small, simple set of libraries. They are built on deeply nested open-source dependency ecosystems in which one package can indirectly support hundreds or thousands of downstream components. The selected problem statement asks for an intelligent risk-analysis system that can map these relationships, explore the downstream consequences of a compromise, identify critical dependencies and affected applications, reveal propagation paths, and rank mitigation priorities.

The supplied research shows why this is difficult. Conventional Software Composition Analysis (SCA) approaches commonly evaluate components as relatively isolated inventory items and rely heavily on static severity measures. This can create alert fatigue while simultaneously hiding structurally important micro-dependencies. The research argues for topology-aware, cross-level analysis that combines ecosystem dependency graphs with fine-grained call-graph reachability. ■filecite■turn1file0■L13-L24■

RippleGuard turns that research direction into an actionable product concept: a software supply-chain digital twin that builds a dependency graph, enriches it with vulnerability and maintenance signals, traces reachable execution paths, simulates compromise scenarios, calculates a contextual Ripple Risk Score, and recommends the intervention that can eliminate the largest amount of downstream risk for the least practical effort.

The core product promise is simple: do not merely report that a dependency is vulnerable; show how the vulnerability can ripple through the ecosystem and tell the team what to fix first.

2. Selected Problem Statement

Open Source Supply Chains: The Ripple Effect

SDG 9 — Industry, Innovation and Infrastructure

Modern software depends on deeply nested open-source packages, meaning a small compromise in a low-level dependency can propagate through many downstream applications. Traditional security tools often evaluate packages individually, making it difficult to understand which components are structurally important or how a compromise could spread through the wider software ecosystem. The real risk of a dependency may therefore be much greater than its own vulnerability score suggests.

Design an intelligent risk-analysis system that maps relationships within a software dependency ecosystem and explores the potential downstream consequences of a compromise. The system should help identify critical dependencies, affected applications, propagation paths, and mitigation priorities while making the reasoning behind its risk assessment visible. Teams may choose how dependency relationships are modeled, how propagation is simulated, how impact is measured, and how mitigation options are ranked. Public open-source dependency information and simulated compromise scenarios may be used.

What the problem is really asking for

#	Requirement	Meaning
1	Map relationships	Represent direct and transitive dependencies as an ecosystem rather than a flat list.
2	Explore downstream consequences	Simulate how a compromised dependency can affect downstream packages and applications.
3	Identify critical dependencies	Find packages whose structural position makes them disproportionately important.
4	Find affected applications	Distinguish theoretical dependency exposure from applications that can actually reach the risky code.
5	Show propagation paths	Make the chain from vulnerable dependency to downstream impact understandable.
6	Prioritize mitigation	Rank actions based on the amount of meaningful risk they can remove.
7	Explain the reasoning	Make the score transparent instead of presenting an unexplained security number.

3. What Existing Approaches Miss

The supplied research compares several vulnerability and prioritization approaches and highlights a recurring issue: different tools answer different parts of the security question, but they do not by themselves provide a unified view of structural exposure, execution reachability, propagation, and actionable mitigation. The comparison identifies CVSS, EPSS, CISA KEV, SSVC, OpenSSF Scorecard and a dynamic propagation approach as having different strengths and failure modes. ■filecite■turn1file0■L255-L373■

3.1 Flat severity is not ecosystem risk

CVSS provides a technical severity score for an isolated flaw. The research notes that this gives no topological awareness and can treat vulnerabilities in unreachable code as high-priority enterprise risks. The central insight for RippleGuard is therefore that severity must be combined with structural exposure and actual reachability. ■filecite■turn1file0■L268-L283■

3.2 Popularity can hide a dangerous micro-dependency

The research describes a 'micro-dependency amplifier effect': small packages can have extremely large transitive fan-in. A vulnerability in such a package can affect many downstream applications even though the package itself may look technically simple. ■filecite■turn1file0■L54-L67■

3.3 Dependency graphs can be misleading

The supplied research highlights the SBOM graph degeneration problem. It reports that many published SBOMs can contain missing relationship information or large numbers of orphan nodes. If a risk engine assumes that an incomplete graph represents reality, it can produce false conclusions about reachability and exposure. ■filecite■turn1file0■L379-L389■

3.4 Reachability is more meaningful than dependency presence

A package can be present in an application's dependency tree without the vulnerable function ever being reachable. The research proposes forward reachability from first-party entry points and backward impact analysis from a vulnerable or modified function. This can reduce false-positive triage by distinguishing theoretical presence from executable paths. ■filecite■turn1file0■L144-L166■

3.5 Maintenance health is usually retrospective

The research identifies a gap between retrospective maintenance indicators and forward-looking abandonment risk. It argues that changes in maintainer activity and maintenance trajectory can become useful early warning signals before security practices degrade. ■filecite■turn1file0■L405-L411■

3.6 Existing analysis can be computationally expensive

Fine-grained function-level reachability offers higher precision but requires more computation and language-specific analysis. The research presents a spectrum from dependency-level analysis to function-level analysis and pre-computed reachability. RippleGuard therefore uses a staged architecture so that a practical MVP can start with lower-cost analysis and progressively add deeper verification. ■filecite■turn1file0■L168-L232■

4. Proposed Solution — RippleGuard

RippleGuard is an **explainable software supply-chain digital twin**. It creates a living model of an application's dependency ecosystem and lets a security or engineering user ask a more useful question than 'Is this package vulnerable?' The system asks: **'If this dependency is compromised, what can actually happen, how far can the ripple travel, and what is the best intervention to stop it?'**

Core workflow

MAP	TRACE	SIMULATE	BREAK
Build dependency and metadata graph.	Verify reachable execution and impact paths.	Model compromise and downstream propagation.	Rank mitigations by risk reduction vs effort.

5. System Architecture

The architecture follows the research direction of combining ingestion and synchronization, topological resolution, program analysis, and prioritization/action. The supplied research explicitly proposes a four-stage pipeline using dependency registries, vulnerability sources, graph construction, call-graph analysis, dynamic scoring, and mitigation queues. [filecite turn1file0L416-L470](#)

Stage	Purpose	Inputs	Outputs
1. Ingestion & Synchronization	Import package manifests/SBOMs and enrich them with dependency, release, vulnerability and repository metadata.	Package registry data, OSV/NVD/CISA KEV, OpenSSF-style posture signals.	Unified package + advisory dataset.
2. Dependency Graph Engine	Resolve versions and construct nodes/edges representing packages, versions and dependency relationships.	Package manifests, lockfiles, registry metadata; graph storage.	Evidence-backed dependency graph.
3. Reachability & Propagation Engine	Trace forward from application entry points and backward from vulnerable targets; simulate propagation through the graph.	Static/pre-computed call graphs plus dependency graph.	Reachable paths, affected applications, propagation paths.
4. Risk & Mitigation Engine	Combine severity, topology, reachability, propagation and business context; rank practical interventions.	Risk features + user-defined criticality/effort.	Ripple Risk Score, mitigation queue, explanation.

6. Detailed Product Features

6.1 Ecosystem Map

The first screen presents the software ecosystem as a graph rather than a flat vulnerability list. Packages become nodes and dependency relationships become edges. Node size or emphasis can represent structural importance, while different states can represent vulnerability, reachability, or business criticality.

Example

```
Application A → Package B → Package C → Vulnerable Package D
Application E → Package C → Vulnerable Package D
Application F → Package X → Package Y
```

This immediately exposes the fact that Package D may be structurally important because several paths converge on it.

6.2 Dependency Criticality Engine

RippleGuard assigns structural importance using graph features such as transitive fan-in, dependency depth, centrality and number of downstream applications. The research emphasizes that scale-free dependency ecosystems contain highly connected hubs and that targeted attacks on central components can have disproportionate effects. [filecite turn1file0 L34-L53](#)

6.3 Explainable Ripple Risk Score

Instead of hiding the result behind a single unexplained number, RippleGuard exposes the contributing signals. A practical MVP can normalize the following dimensions to 0–100 and combine them using explicit weights chosen by the team:

- Vulnerability severity — how technically serious the flaw is.
- Ecosystem exposure — how many downstream components depend on the package.
- Structural centrality — how important the node is to the dependency topology.
- Reachability — whether vulnerable functionality is actually reachable from relevant application code.
- Propagation potential — how many meaningful paths and hops a compromise can traverse.
- Business criticality — how important the affected application/service is to the organization.
- Maintenance posture — signals such as maintainer activity or repository security hygiene.

The research provides a related cross-level formulation in which code-level risk and ecosystem exposure are composed multiplicatively, $R(p) = \Phi \times \psi$. RippleGuard can use that principle as a research-inspired option while keeping the MVP explanation simple and auditable. [filecite turn1file0 L92-L108](#)

6.4 The 'What If?' Compromise Simulator

This is the signature feature. The user selects any package and launches a simulated compromise. The system can model scenarios such as malicious release, maintainer takeover, vulnerability exploitation or package abandonment. The simulator then traverses the dependency graph and calculates downstream impact.

Simulation output: affected packages, affected applications, critical services, propagation depth, number of propagation paths, estimated risk and a visual animation of the ripple.

Example result

TARGET: package-X
Downstream packages: 1,892
Reachable applications: 18
Critical applications: 5
Propagation paths: 23
Ripple Risk: 87/100 – CRITICAL

6.5 Reachability — From 'Present' to 'Actually Used'

Dependency presence alone does not prove that vulnerable functionality is executed. RippleGuard therefore supports staged reachability. A lightweight MVP can begin with import/dependency-level reachability, while a deeper version can use pre-computed or function-level call graphs. The supplied research describes forward reachability from first-party entry points and backward impact analysis from a vulnerable target.

■filecite■turn1file0■L144-L166■

6.6 SBOM Confidence / Graph Health

Before trusting an SBOM-derived graph, RippleGuard checks whether relationship data is complete enough for the requested analysis. It can report node count, edge count, connectedness, orphan nodes and an overall graph-confidence indicator. This directly responds to the research finding that SBOMs can degenerate into flat lists when relationship metadata is missing. ■filecite■turn1file0■L379-L389■

6.7 Ripple Breaker — Mitigation Optimization

After simulation, the system does not stop at impact visualization. It evaluates possible interventions such as upgrading a dependency, removing a redundant dependency, replacing a package, or isolating a service. Each action is compared using estimated risk reduction and implementation effort.

Action	Risk Reduction	Effort	Priority
Patch Application B	64%	Low	1
Patch Application A	18%	Low	3
Replace dependency	52%	Medium	2

The key concept is the Ripple Breaker: identify the smallest practical intervention capable of stopping the largest meaningful portion of the simulated cascade.

6.8 AI Risk Analyst

AI is used as an explanation and decision-support layer rather than as an unsupported security oracle. The graph engine produces structured evidence; the AI converts that evidence into human-readable developer and management explanations. For example, it can explain why a package scored highly, summarize the most important propagation path, and turn a mitigation recommendation into a concise action plan.

Developer view: technical path, package versions, reachable functions and upgrade guidance.

Security/business view: affected services, business criticality, risk reduction and priority.

7. Advanced Features Beyond the Core Problem

7.1 Supply-Chain Time Machine

RippleGuard can compare dependency snapshots across time and visualize how risk changes after dependency updates, newly discovered vulnerabilities, ownership changes, or shifts in maintenance activity. The purpose is to answer not only 'What is risky now?' but also 'Why did our risk increase?'

7.2 Attack Path vs Safe Path

The interface can show the simulated attack path and then replay the graph after a recommended mitigation. Users see which edge or node was effectively neutralized and how much of the ripple disappeared.

7.3 Phantom / Undeclared Dependency Radar

Where analysis data permits, RippleGuard can flag dependencies that appear in runtime or executable usage but are not explicitly represented in the manifest. The research identifies phantom dependencies as a challenge because untracked runtime dependencies can bypass conventional security evaluation.

filecite turn1file0 L233-L249

7.4 Confidence-Aware Risk

Every risk assessment can include an analysis-confidence indicator. For example, a dependency graph with missing relationship data or shallow reachability evidence should not be presented as equally certain as a verified call path. This prevents the system from creating false precision.

8. Example End-to-End Scenario

Stage	Action	What happens
Step 1	Upload	A team uploads package.json, lockfile or SBOM.
Step 2	Map	RippleGuard resolves dependencies and creates the ecosystem graph.
Step 3	Enrich	Packages are enriched with vulnerability and maintenance/security signals.
Step 4	Detect	The system finds a low-level dependency with high fan-in and a serious vulnerability.
Step 5	Trace	Reachability analysis identifies which applications can actually reach the affected functionality.
Step 6	Simulate	The user launches a malicious-release scenario and watches the downstream ripple.
Step 7	Prioritize	The system ranks affected applications by technical and business impact.
Step 8	Break	The Ripple Breaker identifies the intervention with the best risk-reduction/effort trade-off.
Step 9	Explain	The AI Risk Analyst produces developer and management summaries.

9. Feasible Hackathon MVP

The full research problem spans multiple package ecosystems, languages and program-analysis challenges. For a hackathon prototype, the safest approach is to build a narrow but convincing vertical slice rather than claim complete ecosystem-scale coverage.

Recommended MVP scope

- Start with the npm ecosystem.
- Accept package.json / lockfile and optionally a simple SBOM input.
- Build an interactive dependency graph.
- Enrich packages with public vulnerability data.
- Calculate fan-in, depth and centrality.
- Implement dependency/import-level reachability first.
- Implement simulated compromise propagation.
- Generate a Ripple Risk Score with visible factors.
- Implement Ripple Breaker recommendations.
- Add an AI explanation panel.
- Demonstrate one complete attack → impact → mitigation flow.

Suggested technical architecture

Frontend: React / Next.js with an interactive graph visualization.

Backend: Python FastAPI.

Graph: Neo4j or NetworkX for the MVP.

Data: npm registry/dependency metadata plus public vulnerability feeds such as OSV.

Risk engine: Python-based scoring and propagation simulation.

AI layer: LLM-backed explanation over structured risk evidence.

Implementation roadmap

Phase	Focus	Deliverable
Phase 1	Foundation	Input parser, package metadata and graph construction.
Phase 2	Risk graph	Vulnerability enrichment, centrality/fan-in and initial Ripple Risk Score.
Phase 3	Simulation	Compromise scenarios, propagation traversal and blast-radius calculation.
Phase 4	Reachability	Import-level reachability, with a path toward deeper call-graph analysis.
Phase 5	Ripple Breaker	Mitigation candidates, effort/risk-reduction ranking.
Phase 6	Experience	Interactive dashboard, AI explanations and polished demo scenario.

10. How RippleGuard Scores Against the Judging Criteria

10.1 Innovation Beyond Given Requirements

Core requirement: map dependencies, simulate propagation, measure impact and prioritize mitigation.

Beyond the requirement: an interactive software-supply-chain digital twin, What-If compromise simulation, Ripple Breaker optimization, SBOM graph confidence, time-based risk views, attack-vs-safe path replay, and an explainable AI analyst.

10.2 Feasibility of the Solution

The MVP deliberately starts with one ecosystem and a staged reachability model. This keeps the prototype buildable while leaving a clear expansion path toward additional package managers, deeper static analysis and CI/CD integration. The research itself recognizes the cost and language-specific complexity of function-level reachability, making a staged approach more credible. ■filecite■turn1file0■L168-L232■

10.3 Inclusivity and Real Users

RippleGuard serves multiple user perspectives: developers need exact dependency paths and remediation guidance; security engineers need ecosystem-level risk and prioritization; engineering managers need business impact and effort trade-offs; and open-source maintainers need visibility into the structural importance of their packages.

10.4 Marketing / Media Strategy

The primary target audience is enterprise security and DevSecOps teams, followed by development teams and open-source maintainers. The product message is: **'Your most dangerous dependency may not have the highest vulnerability score. See the ripple before it becomes a breach.'**

Go-to-market can begin with an open developer edition for SBOM/dependency analysis, technical content and GitHub/CI integrations, followed by team and enterprise offerings. Demonstrable before/after simulations make the product particularly suitable for technical marketing and conference demonstrations.

10.5 Monetisation Strategy

Tier	Value
Developer / Free	Local or limited repository analysis, basic graph and risk view.
Pro / Team	Continuous monitoring, advanced simulation, mitigation recommendations and collaboration.
Enterprise	Organization-wide dependency graph, RBAC/SSO, compliance reporting, private deployment and CI/CD.
Risk Intelligence API	Expose Ripple Risk and dependency-impact intelligence to existing security platforms.

11. Recommended Demo Narrative

The strongest demonstration should revolve around one dependency and one simulated compromise. The goal is to make the 'Butterfly Effect' visible in under two minutes.

Time	Moment	What to show
0:00–0:15	The problem	Show a normal vulnerability alert and ask: 'What if this small package is used everywhere?'
0:15–0:35	Map	Reveal the dependency graph and highlight the structurally central package.
0:35–0:55	Simulate	Click 'Simulate Compromise' and animate the propagation.
0:55–1:15	Explain	Show reachable applications, critical services, paths and Ripple Risk Score.
1:15–1:35	Break	Show the recommended intervention and the percentage of ripple risk removed.
1:35–1:55	AI explanation	Switch between developer and security-manager explanations.
1:55–2:00	Closing	End with: 'Map. Trace. Simulate. Break.'

12. Final Positioning

RippleGuard should be positioned as an **action-oriented supply-chain risk intelligence platform**, not simply another vulnerability scanner. The system connects four questions that are often handled separately: **Where is the dependency? Can the risky code actually be reached? What happens if it is compromised? What is the smallest action that stops the largest ripple?**

The strongest innovation is therefore not any single algorithm. It is the seamless combination of ecosystem topology + reachability + propagation simulation + explainable risk + mitigation optimization inside one interactive workflow.

In one line: RippleGuard turns dependency security from a list of vulnerabilities into a simulation of consequences and a plan for breaking the cascade.

Research basis

This document is grounded primarily in the supplied research document, which covers structural topology, micro-dependency amplification, dependency resolution, reachability, propagation scoring, SBOM graph degeneration, existing scoring frameworks, and a four-stage architecture for intelligent supply-chain risk analysis. Key claims above are cited to the supplied research where applicable.